

JVM Architecture Explained

*ClassLoader • Memory Areas • Heap
• Stack • Garbage Collection*

"Understand the engine that runs every Java program."



What is JVM?

JVM = Java Virtual Machine

It is the runtime engine responsible for :

- ✓ Running Java bytecode
- ✓ Managing memory
- ✓ Ensuring platform independence
- ✓ Handling garbage collection
- ✓ Managing multithreading

“Write Once, Run Anywhere — because JVM handles the system differences.”



Components of JVM

JVM mainly consists of :

- Class Loader Subsystem

- Runtime Data Areas

- Execution Engine

- Native Method Interface & Libraries

These work together to load, verify, execute, and manage your Java code.



ClassLoader Subsystem

Responsible for loading .class files into JVM.

Types of Class Loaders :

1. Bootstrap ClassLoader

→ Loads core Java classes (java.lang, java.util)

2. Extension ClassLoader

→ Loads extension libraries (JDK/ext)

3. Application ClassLoader

→ Loads classes from your project / classpath

Follow Hierarchy

Child → asks Parent first: Parent Delegation Model

Prevents malicious overriding of Java core classes.



Runtime Data Areas (Memory Model)

JVM divides memory into multiple areas :

- 📌 **Heap** (Objects)
- 📌 **Stack** (Method calls)
- 📌 **Method Area** (Class metadata)
- 📌 **PC Register**
- 📌 **Native Method Stack**

Each plays a specific role in program execution.

Heap Memory (Where Objects Live!)

- ✓ Stores objects and instance variables
- ✓ Shared across all threads
- ✓ Garbage Collector works here

Heap is divided into :

1. Young Generation

- Eden space
- Survivor space S1, S2
- (Objects first created here)

2. Old Generation

(Long-living objects promoted here)

Most interview questions come from Heap + GC.

Stack Memory (Per-thread memory)

Each thread has its own **Stack** :

Contains:

- ✓ local variables
- ✓ method call frames
- ✓ return values
- ✓ references (not objects)

Fastest memory in JVM.

Removed automatically when method completes (LIFO order).



Method Area + MetaSpace

Method Area (Pre-Java 8: PermGen)

Stores :

- ✓ Class metadata
- ✓ Method bytecode
- ✓ Static variables
- ✓ Constant pool

Java 8+ uses **MetaSpace** (stored in native memory) → fixes PermGen OutOfMemory errors.



Garbage Collection (GC)

GC automatically removes unused objects from the Heap.

🔥 **GC Works in Two Steps :**

1. Marking Phase

Finds objects referenced/reachable

2. Sweeping Phase

Deletes unreferenced objects

GC Types :

- ✓ Serial GC
- ✓ Parallel GC
- ✓ CMS
- ✓ G1 GC (Default for modern JVMs)

G1 GC = Low pause time + Better performance



Execution Engine

Responsible for running your bytecode.

Components:

- ✓ **Interpreter** → Executes line by line
- ✓ **JIT Compiler** → Converts hotspots to native code for speed
- ✓ **Profiler** → Identifies frequently executed code

JIT improves performance by 10x in long-running applications.



Putting It All Together

ClassLoader loads class

Bytecode stored in Method Area

Objects created in Heap

Methods execute in Stack

GC cleans unused objects

Execution Engine runs the code

This is how JVM powers every Java application.



Interview Questions

Q1: Difference between Heap and Stack?

→ Heap stores objects; Stack stores method frames.

Q2: What is Parent Delegation Model?

→ Child classloader delegates loading to parent first.

Q3: What is PermGen vs MetaSpace?

→ MetaSpace uses native memory; PermGen had fixed size.

Q4: What triggers Garbage Collection?

→ Not programmer — JVM decides based on memory pressure.

Q5: What is Young Gen vs Old Gen?

→ Short-lived vs long-lived objects.



Summary

- ✓ JVM is the heart of Java
- ✓ ClassLoader loads classes
- ✓ Heap stores objects, Stack stores execution
- ✓ GC manages memory cleanup
- ✓ JIT optimizes performance

Understanding JVM = Writing efficient, optimized, scalable Java code. 🚀

Follow for More 